Lecture 13
Procedures and Managing Data

Comp_Sci 213

Introduction to Computer Systems

Monday, Nov. 9, 2026

Kate Smith – Fall 2026

Slides adapted from:
Ghena, St-Amour, Hardavellas, Bustamente (Northwestern), Bryant, O'Hallaron (CMU), Garcia, Weaver (UC Berkeley)

More about data alignment

K	Types
1	char
2	short
4	int, float
8	long, double, char *

- Alignment based on the principle that a primitive of K bytes must have an address that is a multiple of K
- Padding inserted to help with this in structs
  - Entire struct size must also be a multiple of its largest element
  - Good rule for ordering struct elements: go from largest to smallest!
- Virtual memory uses alignment as well
  - Memory is managed in 4 KB pages
  - Aligned data (primitives) never straddle two pages

Topics

- Procedures
  - **Managing Local Data**
- Register saving
  - Recursion Example

Procedures Recap

- ***Procedures*** provide a way to package code that implements some functionality with designated set of arguments and an optional return value
  - Appear as functions, methods, subroutines, handlers, etc. in different programming languages
- When procedure P (***caller***) calls procedure Q (***callee***), we see:
  - *Passing control* – the program counter must be set to the starting address of the code Q and then set to the instruction in P following the call to Q on return
  - *Passing data* – P must be able to provide one or more parameters to Q, and Q must be able to return a value back to P
  - *Allocating and deallocating memory* – Q may need to allocate space for local variables when it begins then free that storage before it returns

Call-Local State

- Need some place to store state for each call
  - Return address
  - Arguments
  - Local variables
  - Temporary space (if needed)

- Note: these are separate for each call, not each function
  - Function could be called recursively, but each call needs its own local variables

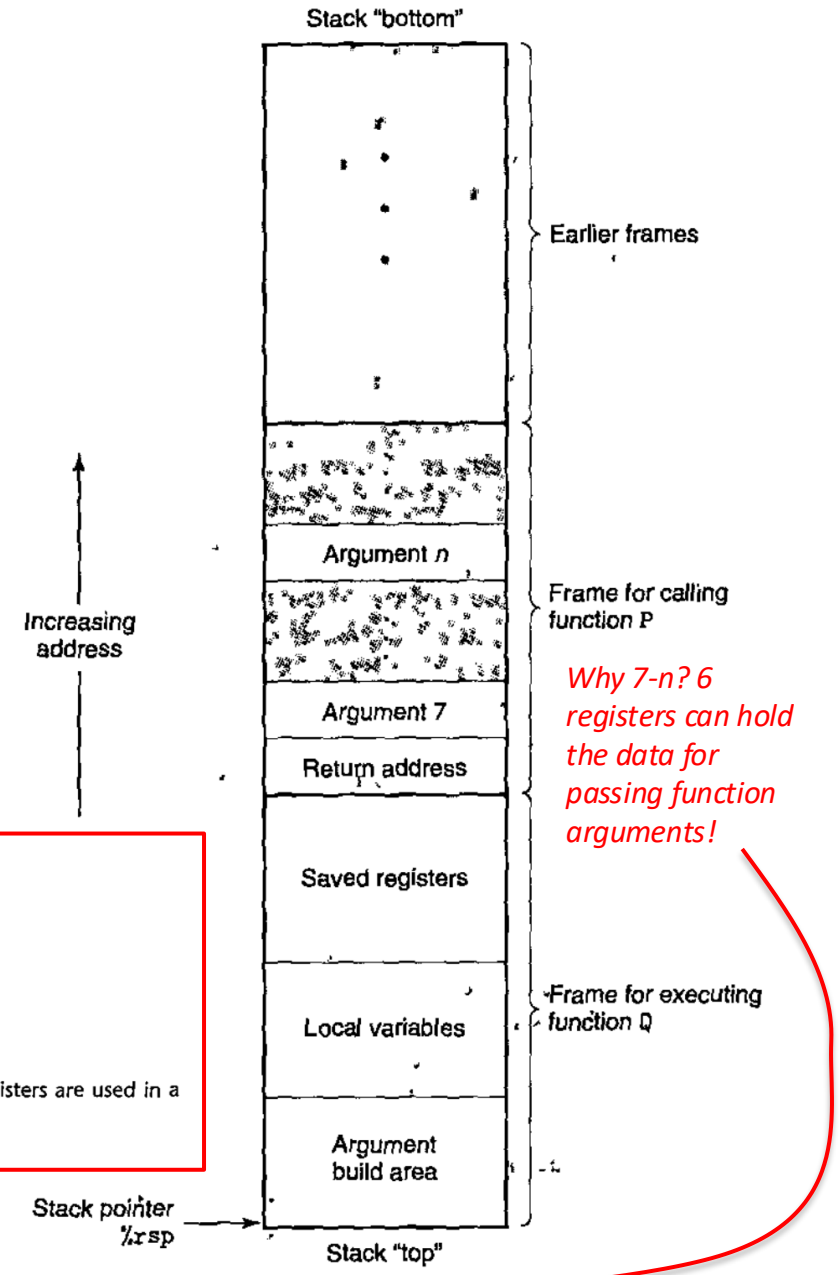
- State only needs to exist until the function returns

Using the Stack for Call-Local State

- Place local state on the stack
- Stack allocated in **Frames**
  - Frame = State for a single procedure invocation
  - Allocated by "setup" code at the start of function
  - Deallocated by "teardown" code before returning
- Caller functions runs the Callee function
  - **Caller** – function that called this function
    - Its state is in the "previous frame"
  - **Callee** – the currently running function
    - Its state is in the "current frame"

Remember: stack is last in, first out!

Figure 3.25 **General stack frame structure.** The stack can be used for passing arguments, for storing return information, for saving registers, and for local storage. Portions may be omitted when not needed.



Operand size (bits)	Argument number 1	2	3	4	5	6
64	%rdi	%rsi	%rdx	%rcx	%r8	%r9
32	%edi	%esi	%edx	%ecx	%r8d	%r9d
16	%di	%si	%dx	%cx	%r8w	%r9w
8	%dil	%sil	%dl	%cl	%r8b	%r9b

Figure 3.28 **Registers for passing function arguments.** The registers are used in a specified order and named according to the argument sizes.

Call Chain Example

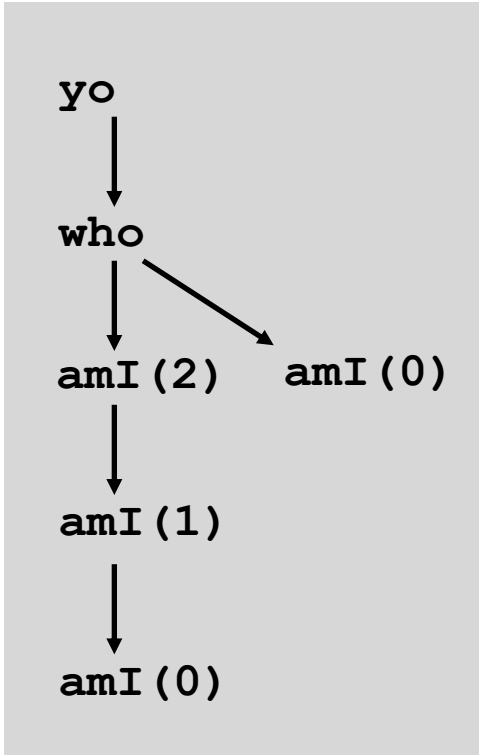
```
yo(…)
{
   •
   •
   who();
   •
   •
}
```

```
who(…)
{
   • • •
   amI(2);
   • • •
   amI(0);
   • • •
}
```

```
amI(int x)
{
   •
 if(x)
   amI(x-1);
   •
   •
}
```

Procedure `amI()` is recursive

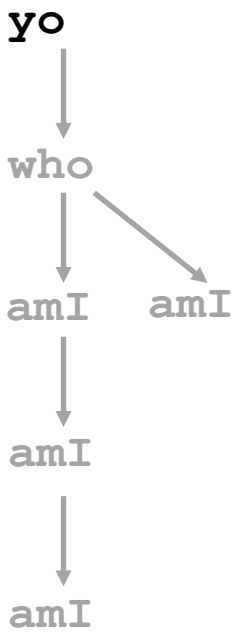
Example Call Chain



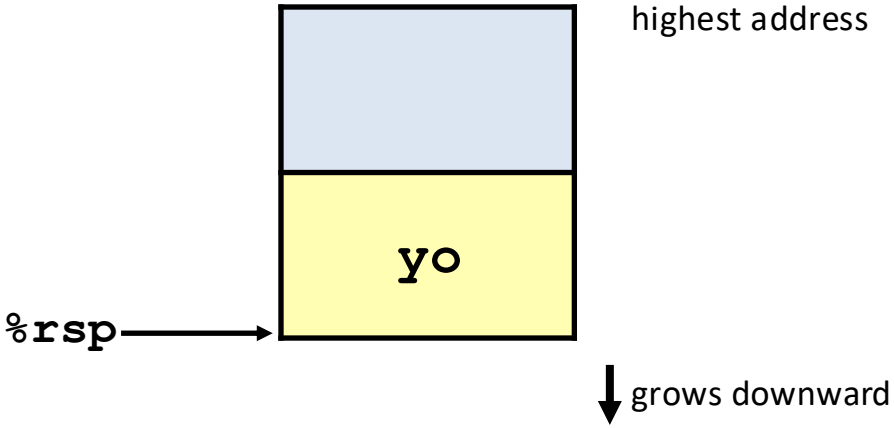
Example

```
yo(...)
{
  •
  •
  who();
  •
  •
}
```

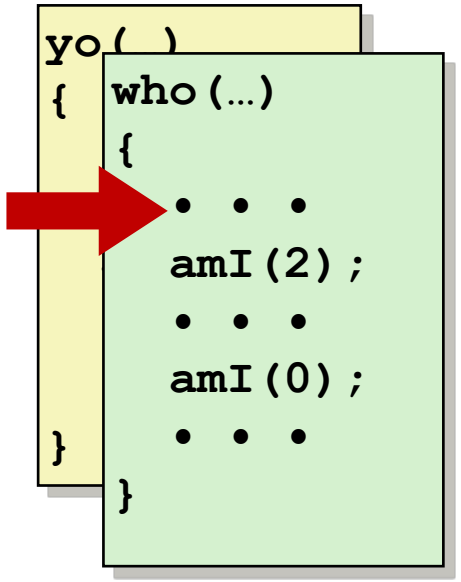
Call Chain

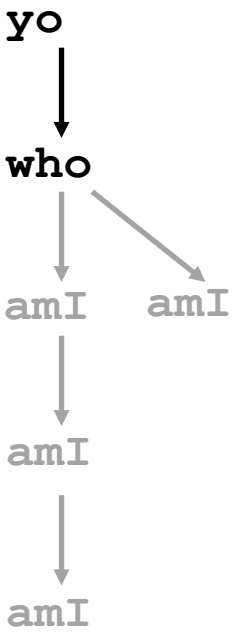


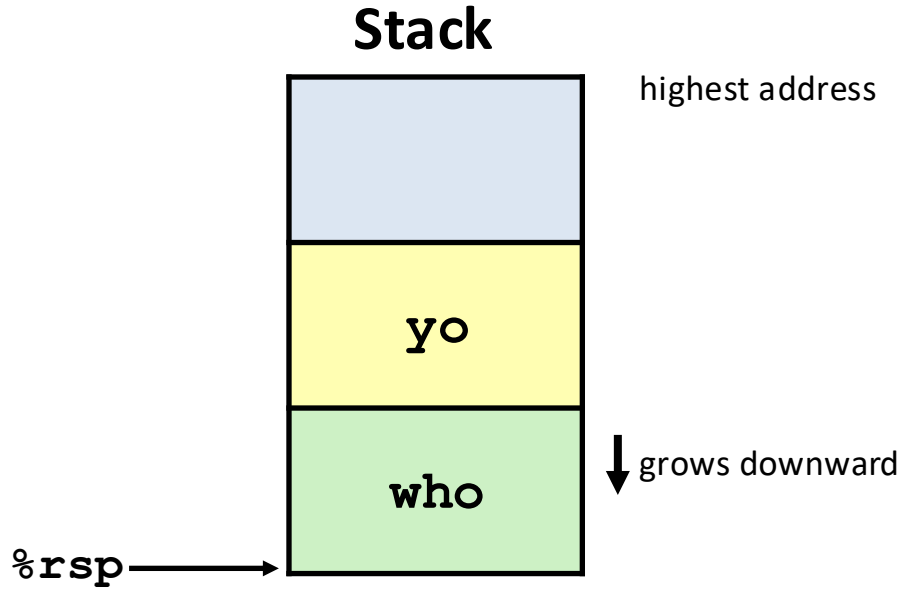
Stack



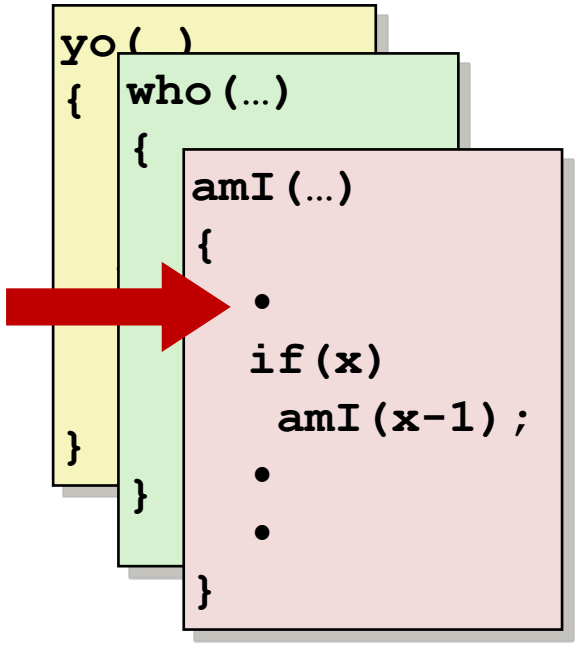
Example

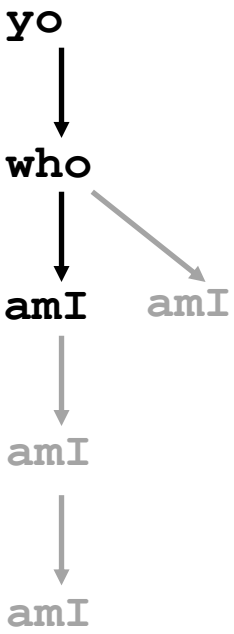

yo( )
{
}
who(…)
{
• • •
amI(2);
• • •
amI(0);
• • •
}

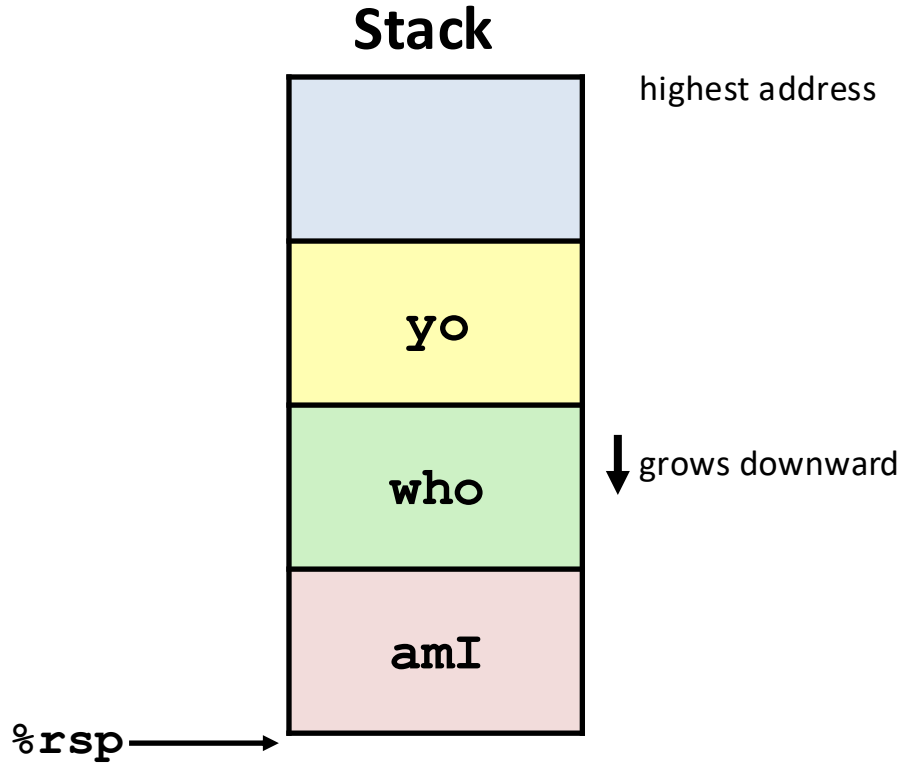

yo
who
amI
amI
amI
amI


Stack
highest address
yo
who
grows downward
%rsp

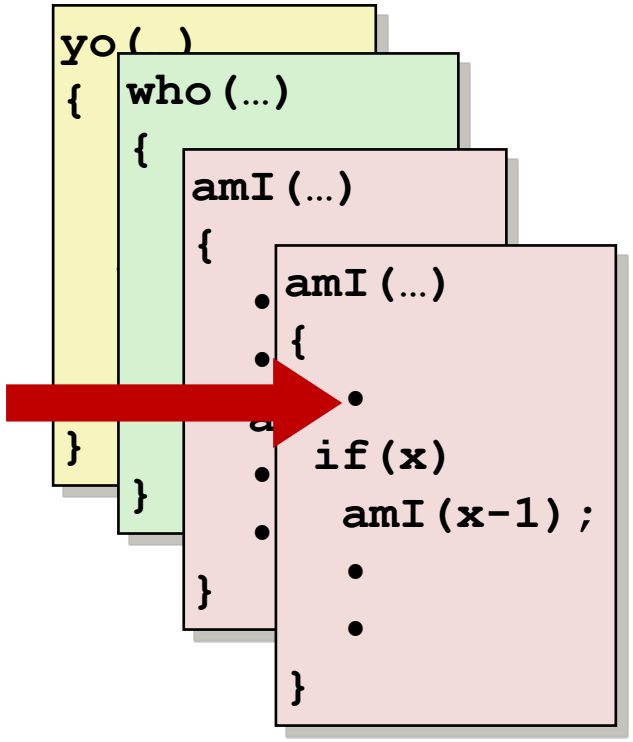
Example

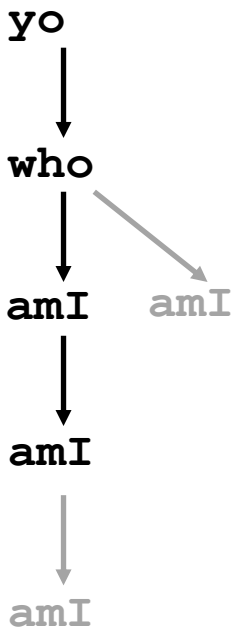

yo( )
{
who(…)
{
amI(…)
{
•
if(x)
amI(x-1);
•
•
}
}
}

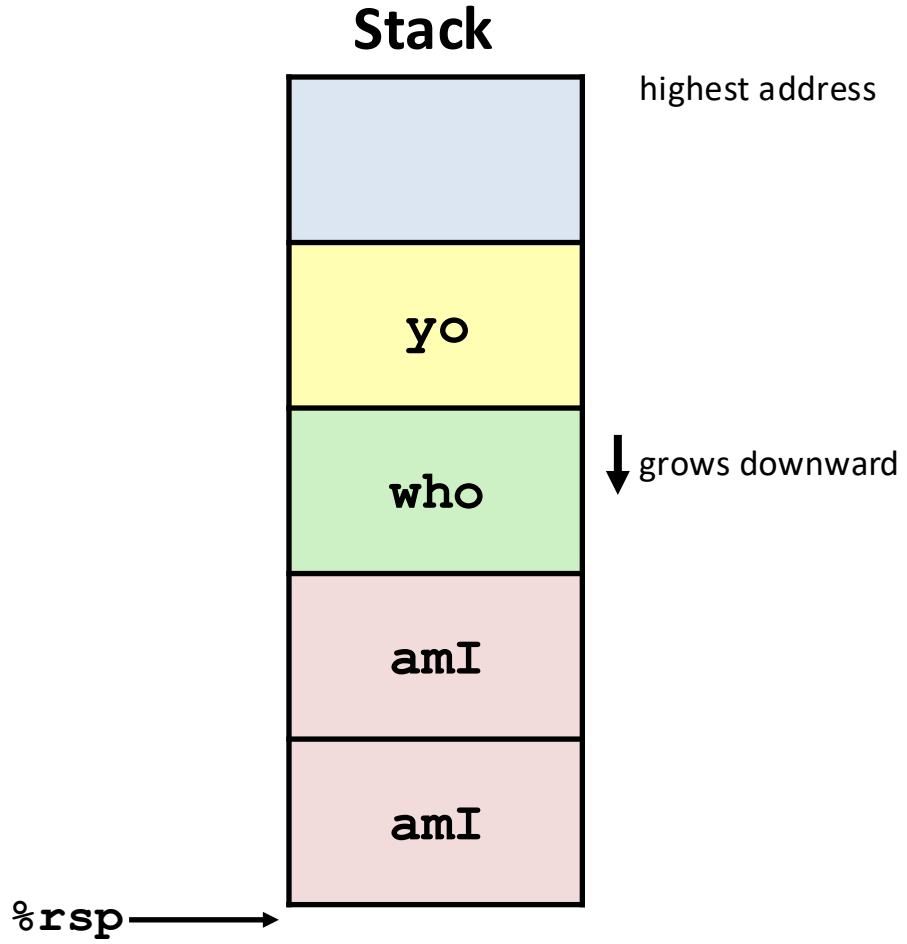

yo
who
amI
amI
amI
amI


Stack
highest address
yo
who
amI
grows downward
%rsp

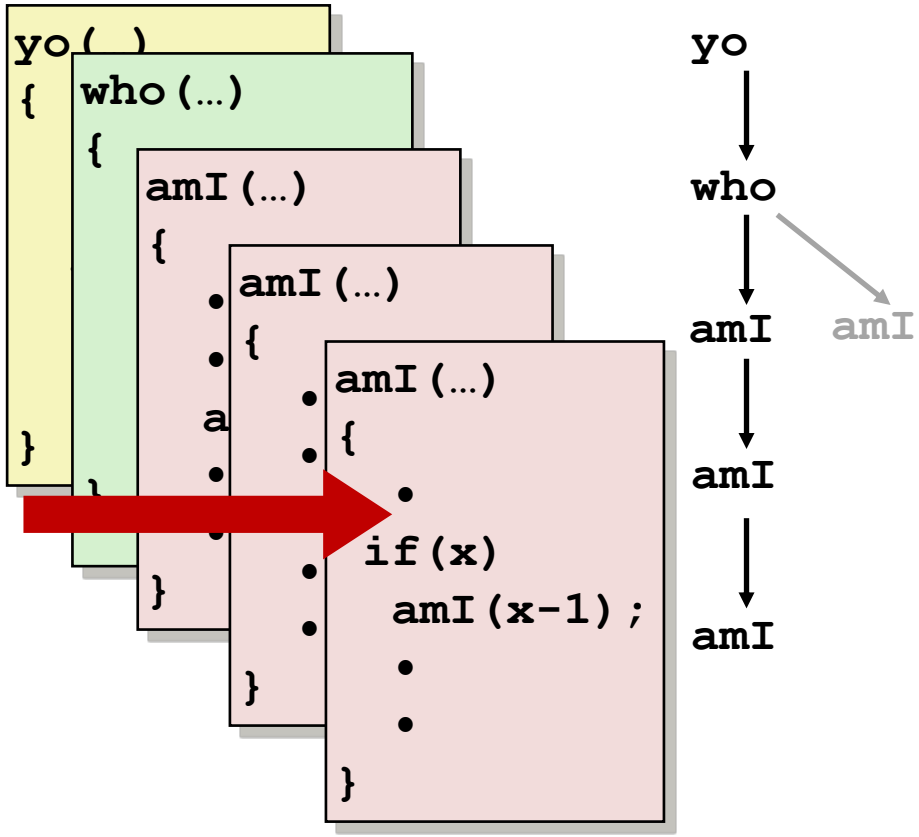
Example

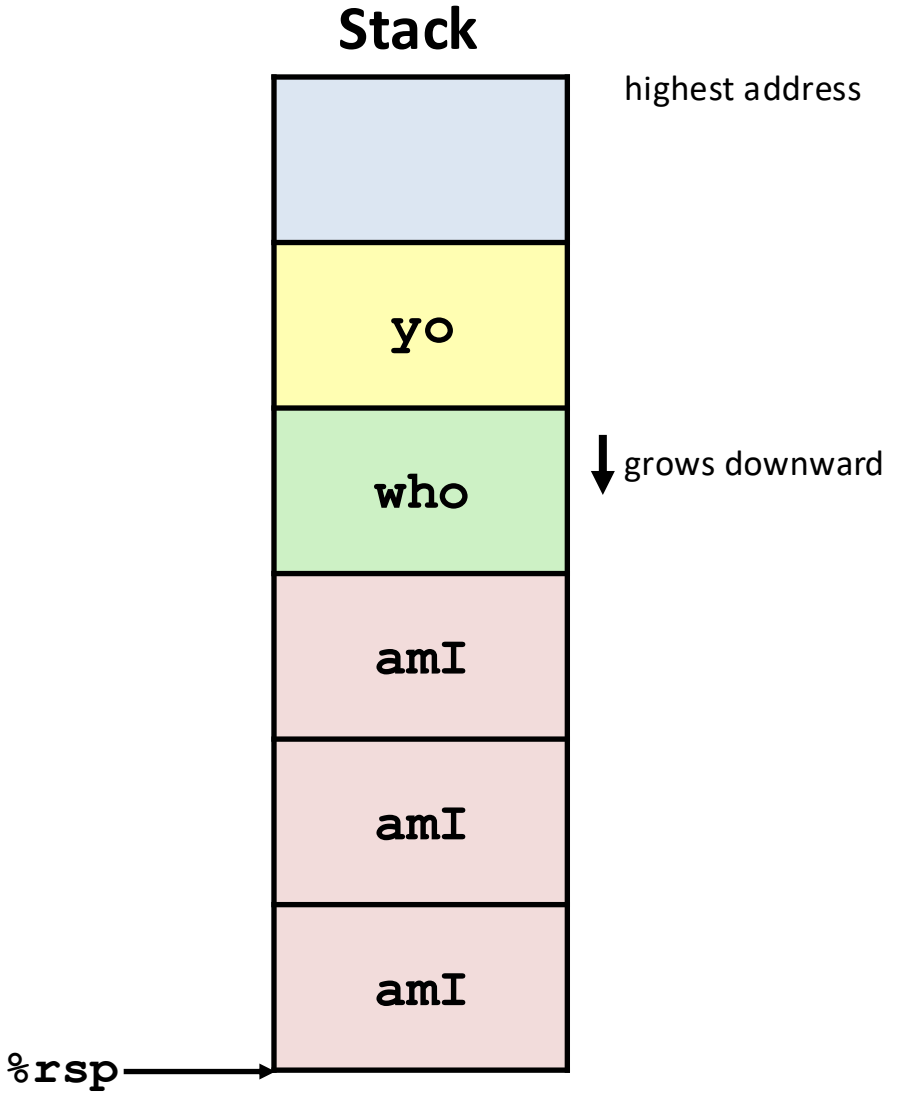

yo( )
{
who(…)
{
amI(…)
{
amI(…)
{
if(x)
amI(x-1);
}
}
}
}


yo
who
amI
amI
amI
amI

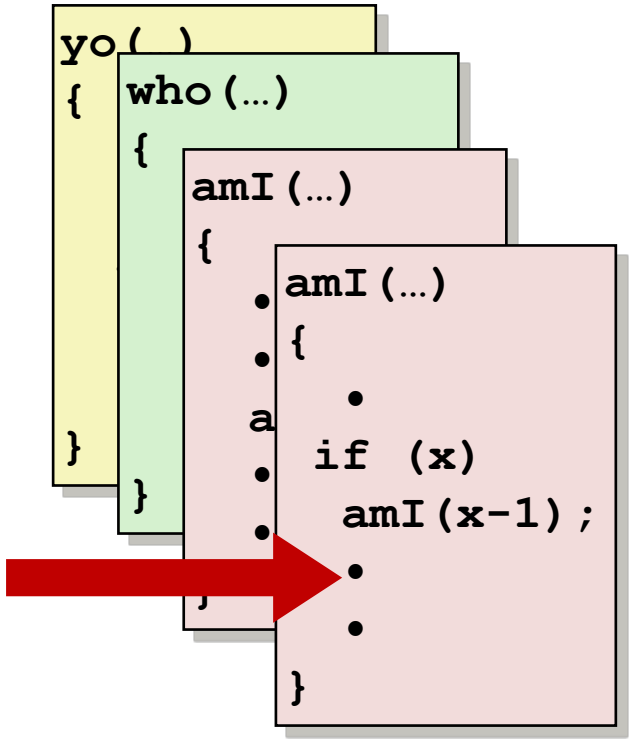

Stack
highest address
yo
who
amI
amI
grows downward
%rsp

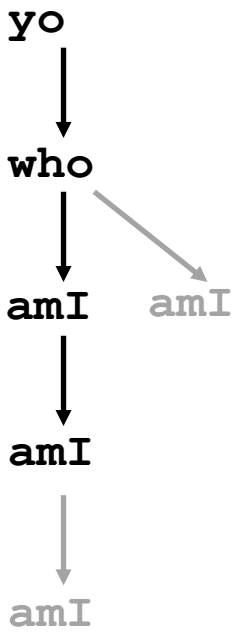
Example

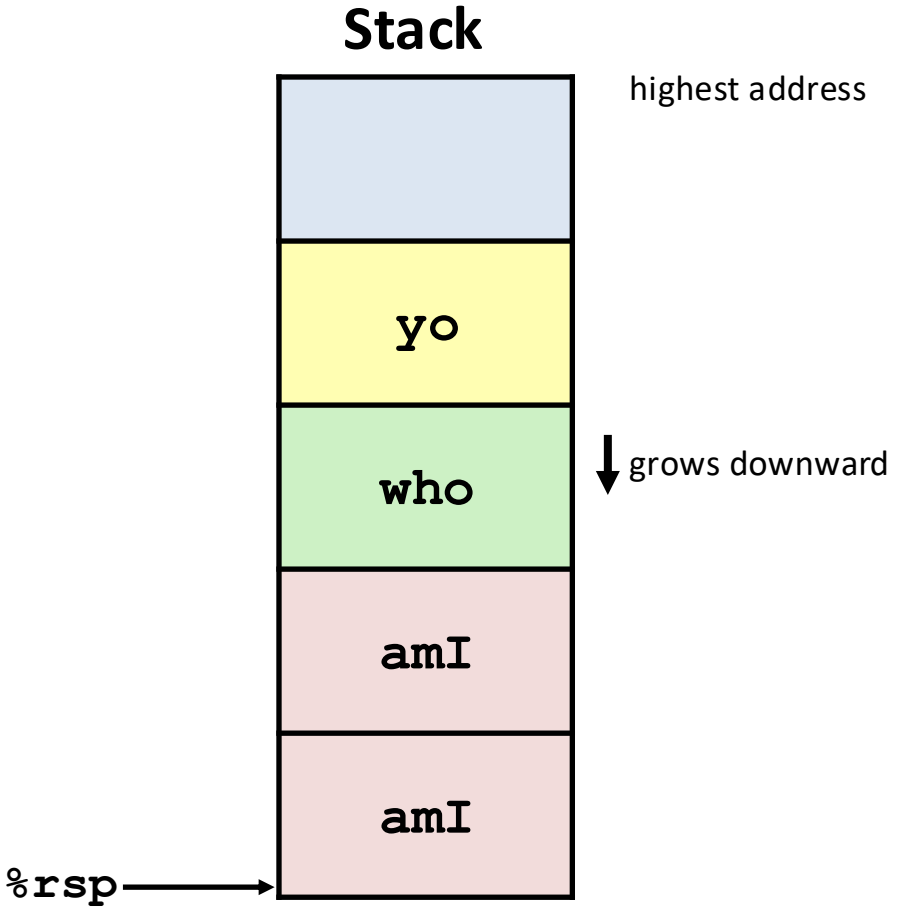

yo( )
{
who(...)
{
amI(...)
{
amI(...)
{
amI(...)
{
if(x)
amI(x-1);
}
}
}
}
yo
who
amI
amI
amI
amI


Stack
highest address
yo
who
amI
amI
amI
grows downward
%rsp

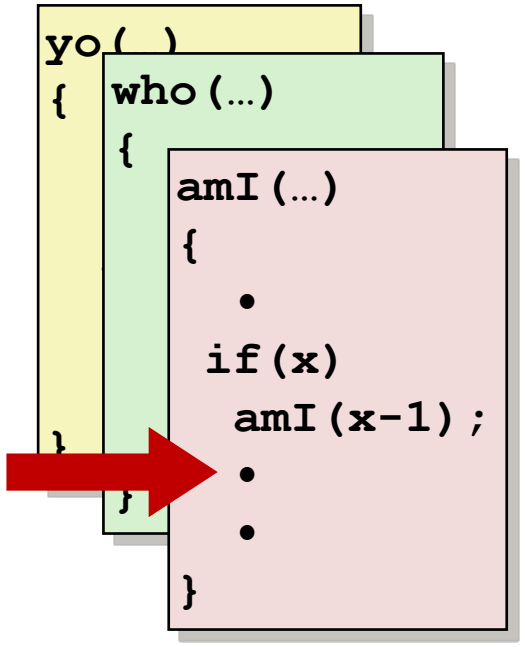
Example

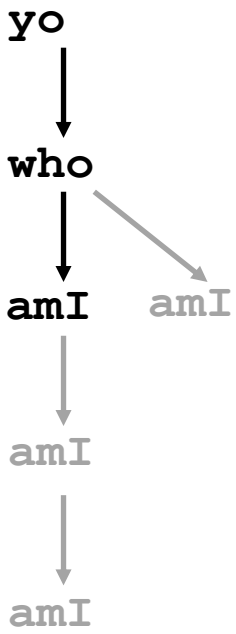

yo( )
{
who(…)
{
amI(…)
{
amI(…)
{
if (x)
amI(x-1);
}

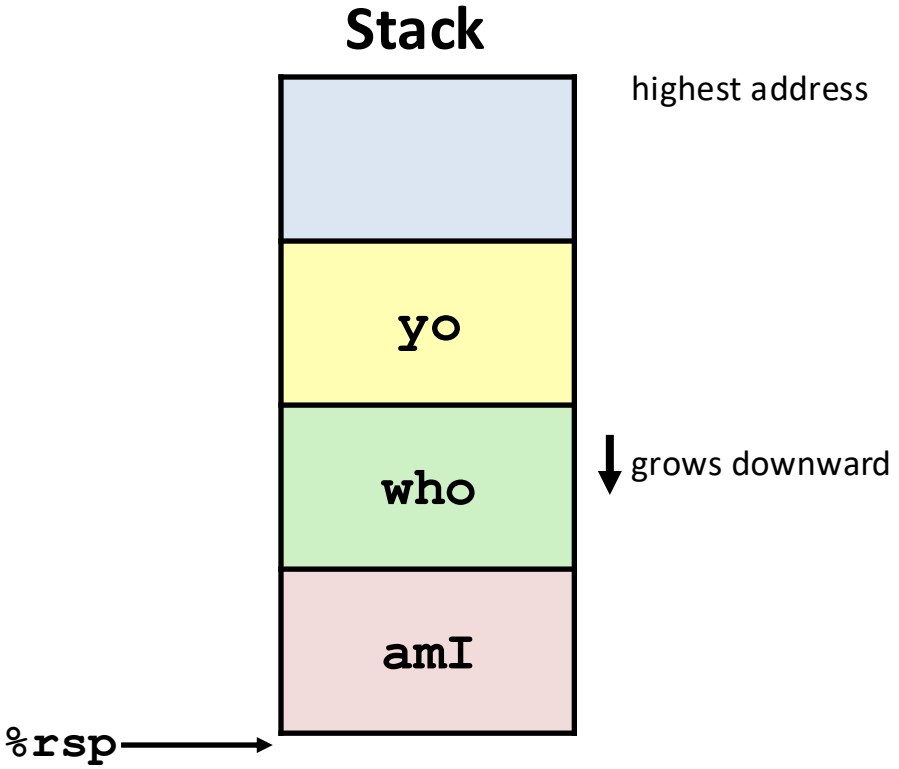

yo
who
amI
amI
amI
amI


Stack
highest address
yo
who
grows downward
amI
amI
%rsp

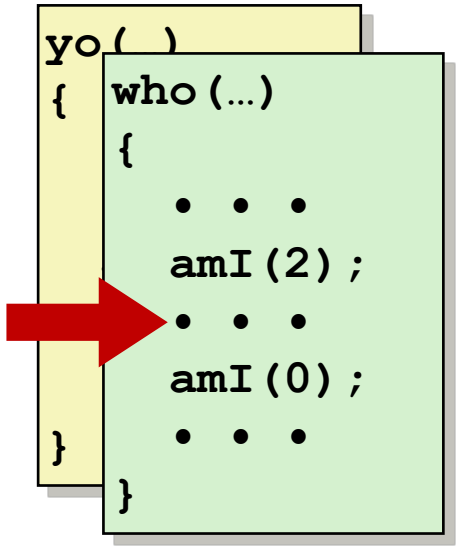
Example

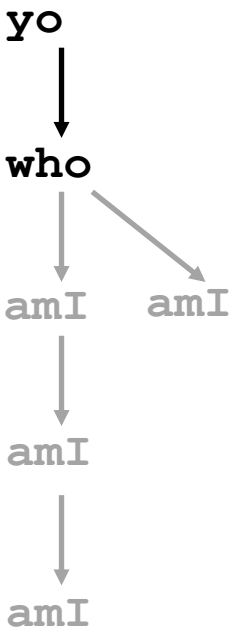

yo( )
{
who(…)
{
amI(…)
{
•
if(x)
amI(x-1);
•
•
}
}
}

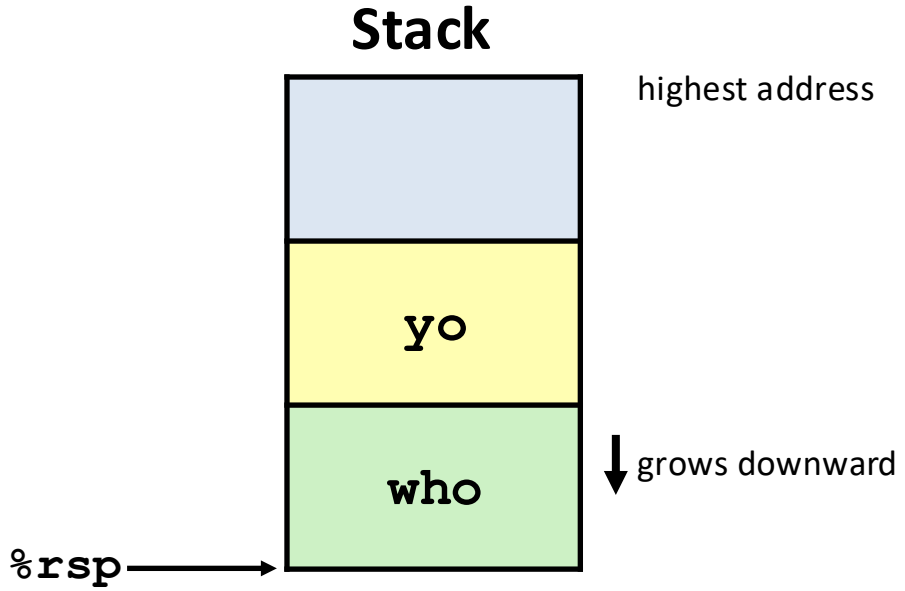

yo
who
amI
amI
amI
amI


Stack
highest address
yo
who
amI
grows downward
%rsp

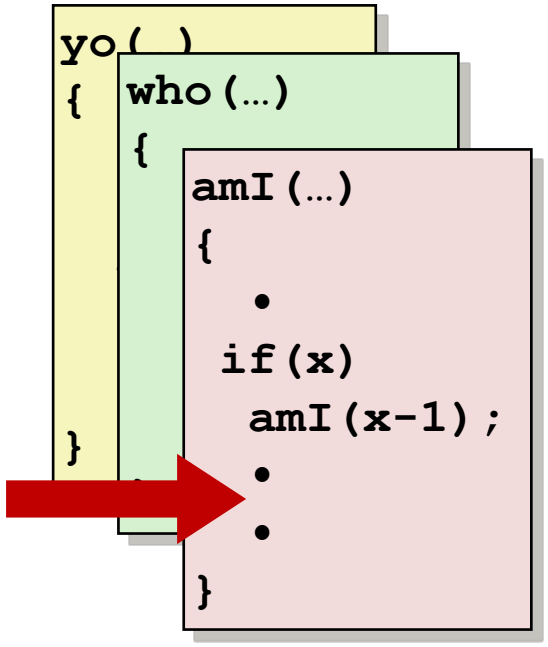
Example

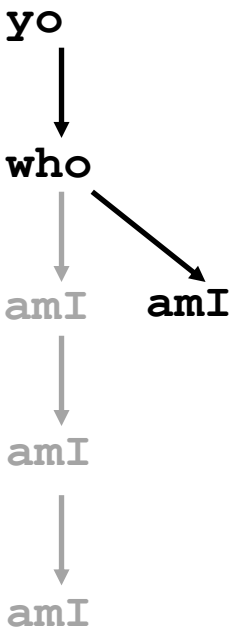

yo( )
{
}
who(…)
{
• • •
amI(2);
• • •
amI(0);
• • •
}

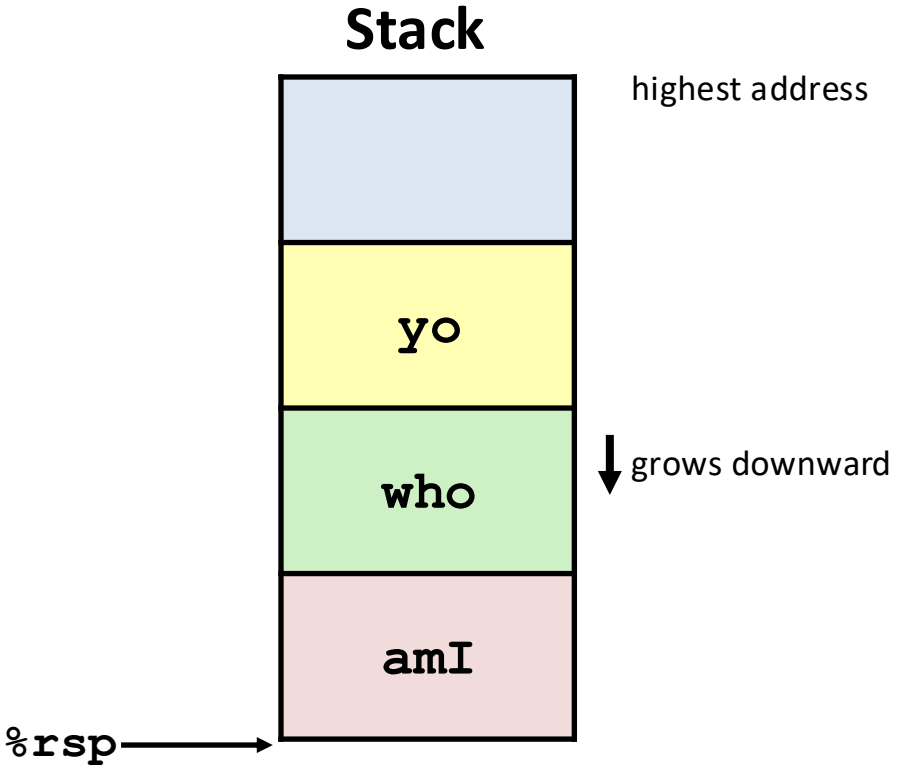

yo
who
amI
amI
amI
amI


Stack
highest address
yo
who
grows downward
%rsp

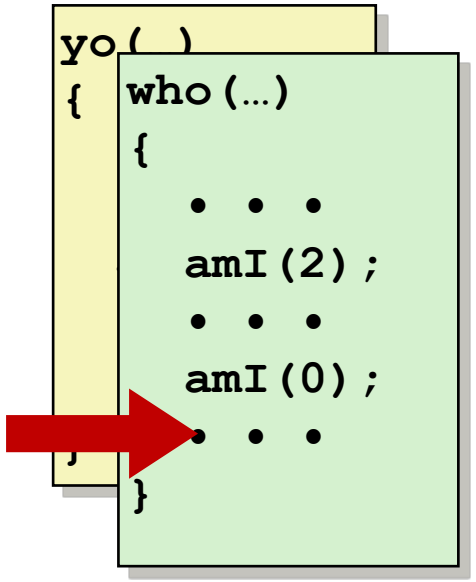
Example

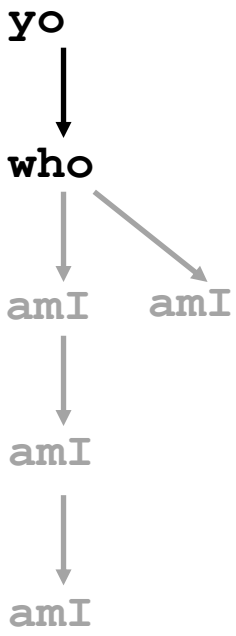

yo( )
{
who(…)
{
amI(…)
{
•
if(x)
amI(x-1);
•
•
}
}

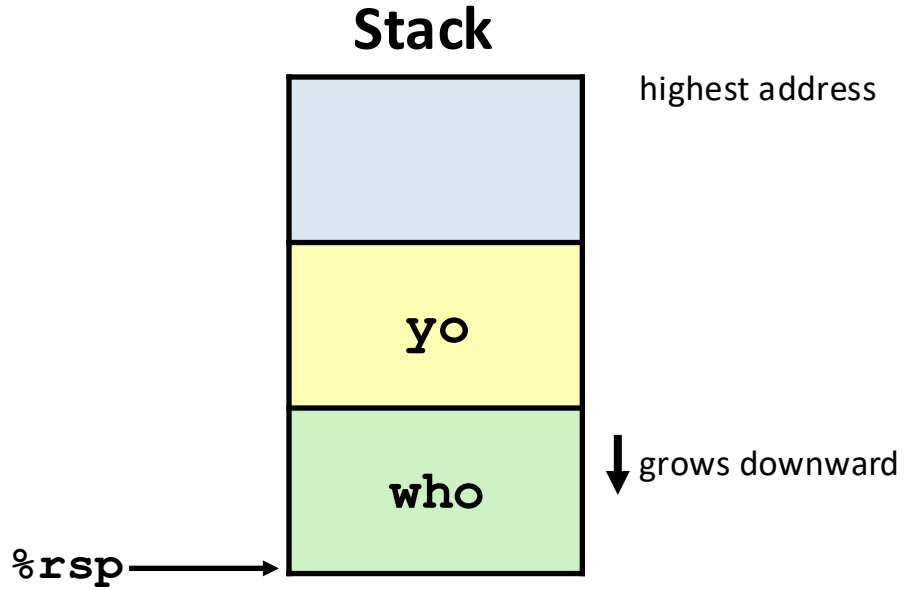

yo
who
amI
amI
amI
amI


Stack
highest address
yo
who
amI
grows downward
%rsp

Example

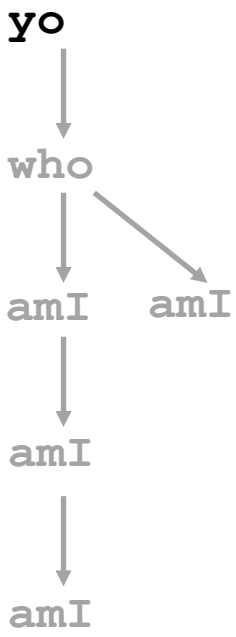

yo( )
{
who(…)
{
• • •
amI(2);
• • •
amI(0);
• • •
}

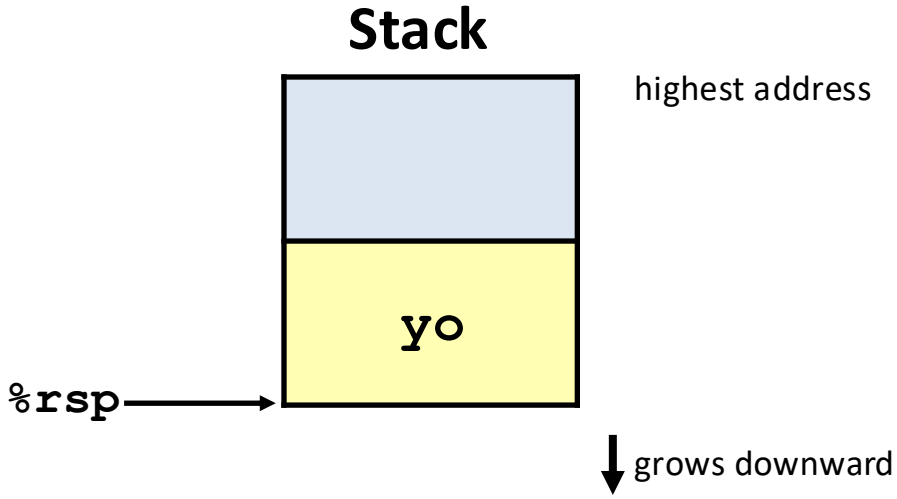

yo
who
amI
amI
amI
amI


Stack
highest address
yo
who
grows downward
%rsp

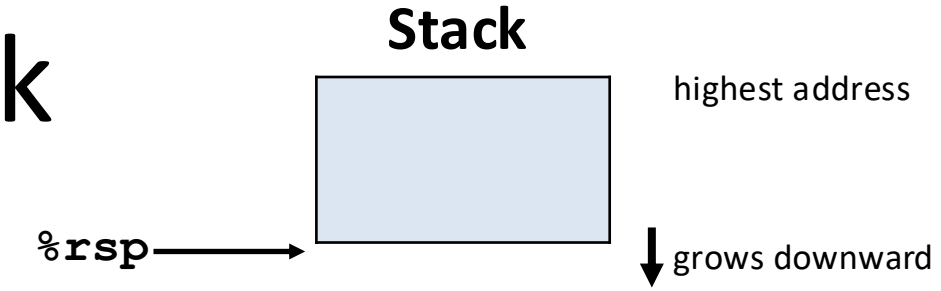
Example

```
yo(...)
{
   •
   •
   who();
   •
   •
}
```





Returning to original stack

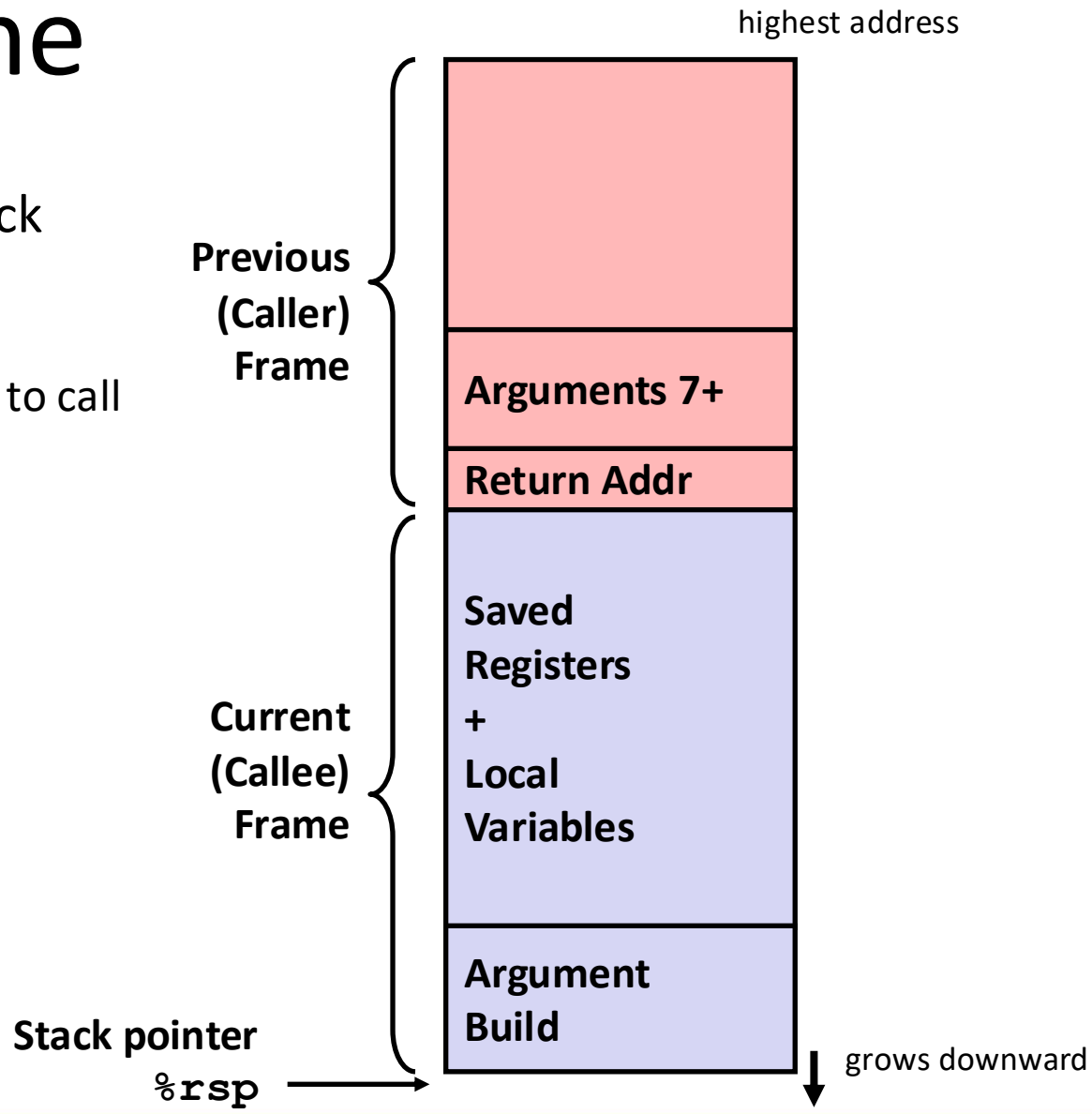


- Stack always eventually returns to its default state
  - Happens automatically in higher-level languages like C
  - Need to manage that ourselves if writing assembly

- Or the program can exit early from anywhere
  - Entire stack is deallocated when the program ends

x86-64/Linux Stack Frame

- Walking from `%rsp` backwards through the stack
- Current (**Callee**) Stack Frame
  - "Argument build": Arguments for function we're about to call
    If there are 7+ and they need to be on the stack
  - Local variables: If we can't keep them in registers
  - Saved register context (we'll get to that soon)
- Previous (**Caller**) Stack Frame
  - Return address
    - Pushed by **`call`** instruction
  - Arguments for callee (current function)

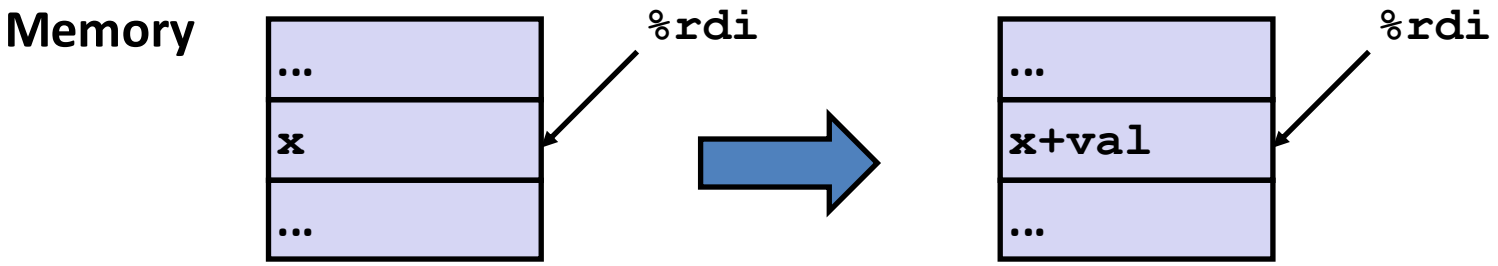


Example: `incr`

```
long incr(long* p, long val) {
    long x = *p;
    long y = x + val;
    *p = y;
    return x;
}
```

```
incr:
  movq    (%rdi), %rax  # x = *p
  addq    %rax, %rsi    # y = x+val
  movq    %rsi, (%rdi)  # *p = y
  ret
```

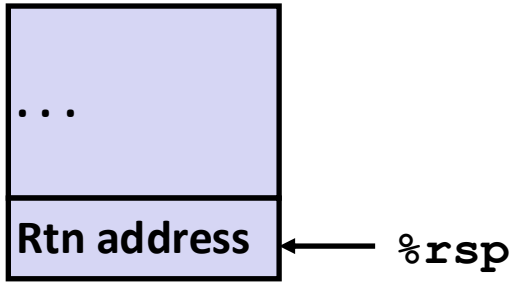
Register	Use(s)
`%rdi`	Argument **`p`**
`%rsi`	Argument **`val`**, also **`y`**
`%rax`	**`x`**, Return value



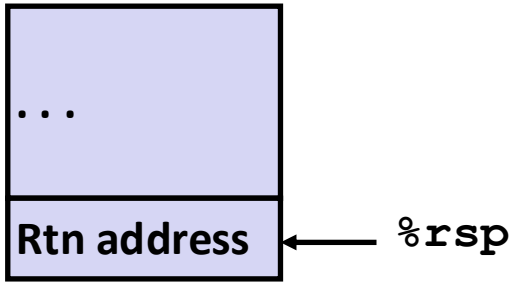
Example: Calling `incr` #1 (local variables)

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

Initial Stack Structure



Resulting Stack Structure



Example: Calling `incr` #1 (local variables)

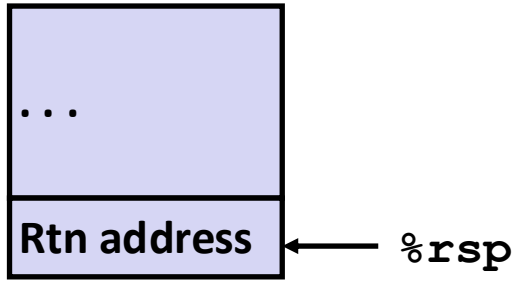
We take **v1**'s address, so must be in memory

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

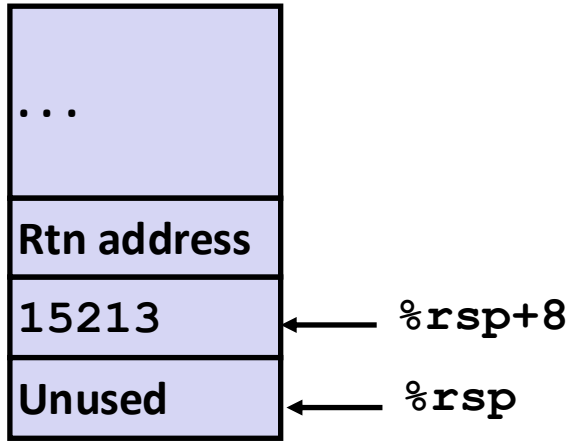
Stack pointer must be multiple of 16

```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movq    $3000, %rsi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Initial Stack Structure



Resulting Stack Structure



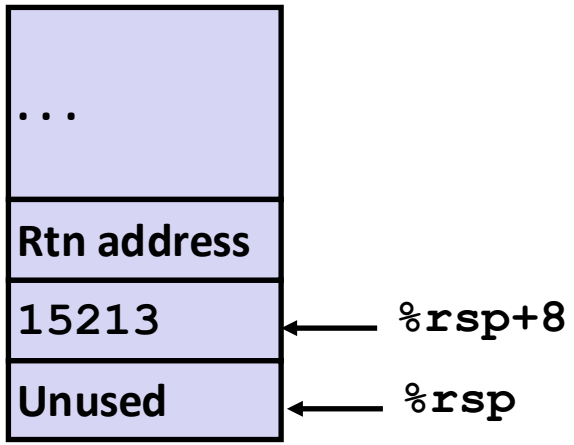
Example: Calling `incr` #2 (argument build)

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

Register	Use(s)
%rdi	&v1
%rsi	3000

```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movq    $3000, %rsi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Stack Structure



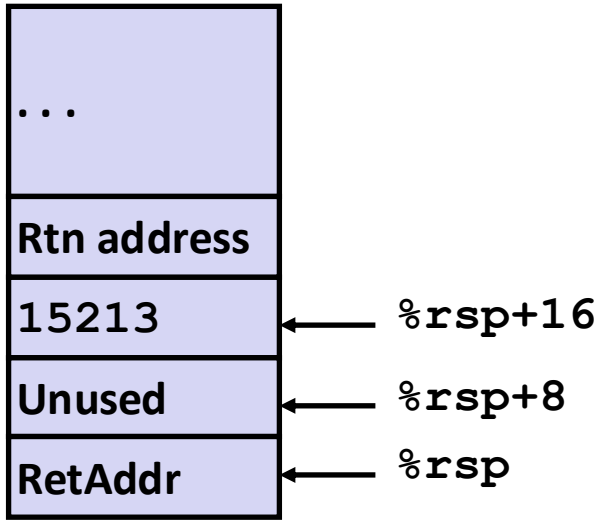
Example: Calling `incr` #3 (control transfer)

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

Register	Use(s)
%rdi	&v1
%rsi	3000

```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movq    $3000, %rsi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Stack Structure



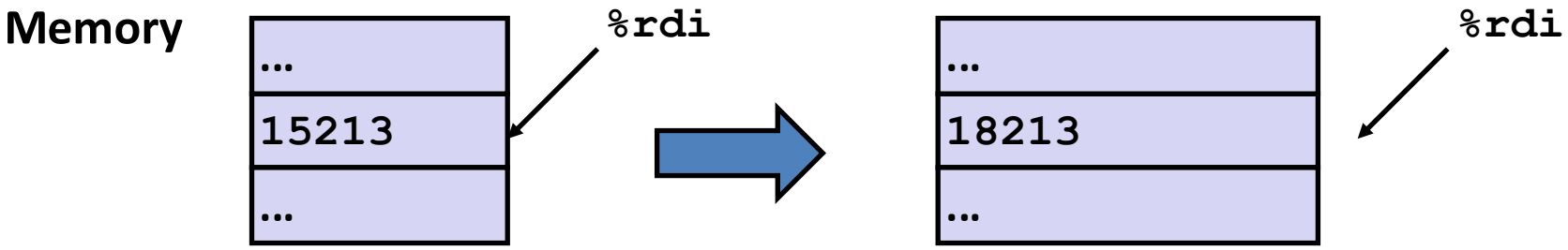
Example: executing `incr`

```
long incr(long *p, long val) {
    long x = *p;
    long y = x + val;
    *p = y;
    return x;
}
```

```
incr:
  movq    (%rdi), %rax
  addq    %rax, %rsi
  movq    %rsi, (%rdi)
  ret
```

Register	Use(s)
%rdi	Argument p
%rsi	Argument val (3000)
%rax	...

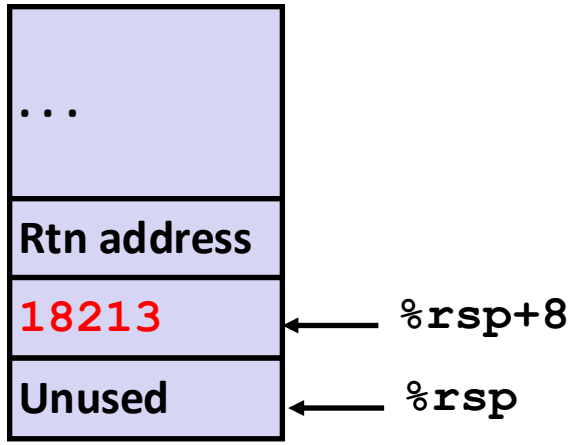
Register	Use(s)
%rdi	Argument p
%rsi	18213 (overwritten, that's fine)
%rax	15213 (return value)



Example: right after executing `incr`

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

Stack Structure




```
call_incr:
   subq     $16, %rsp
   movq     $15213, 8(%rsp)
   movq     $3000, %rsi
   leaq     8(%rsp), %rdi
   call     incr
   addq     8(%rsp), %rax
   addq     $16, %rsp
   ret
```

Register	Use(s)
%rdi	&v1
%rsi	18213
%rax	15213

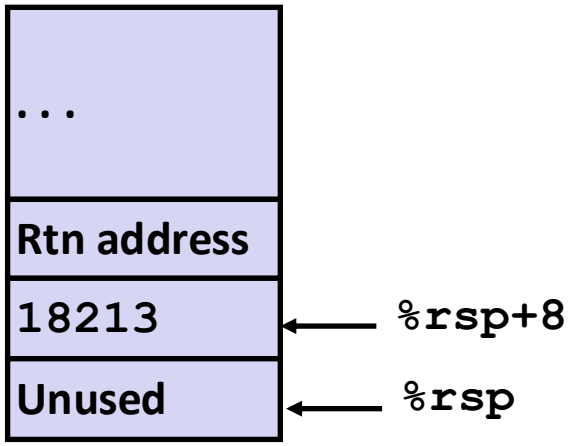
Question: where do we find the return value of `incr`?

Example: Calling `incr` #4 (do the addition)

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movq    $3000, %rsi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Previous stack Structure



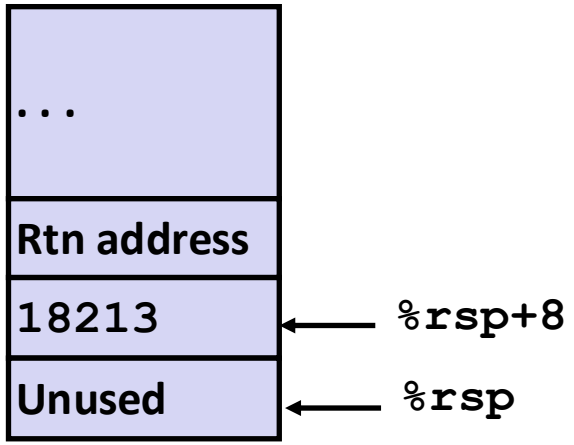
Register	Use(s)
%rax	Return value

Example: Calling `incr` #5 (cleanup)

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

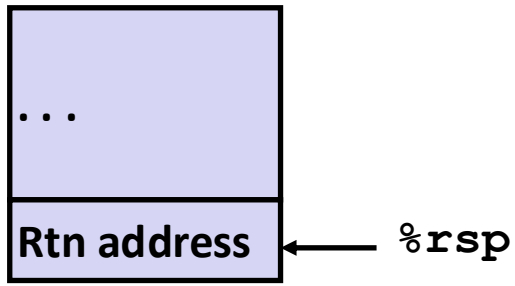
```
call_incr:
  subq    $16, %rsp
  movq    $15213, 8(%rsp)
  movq    $3000, %rsi
  leaq    8(%rsp), %rdi
  call    incr
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Previous stack Structure



Register	Use(s)
%rax	Return value

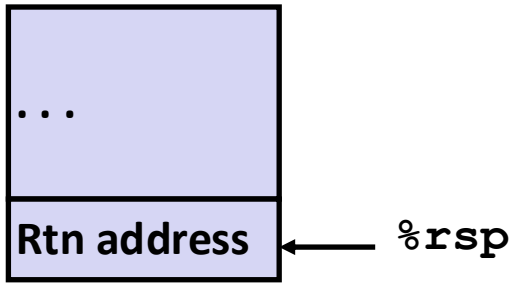
Updated Stack Structure



Example: Calling `incr` #6

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1+v2;
}
```

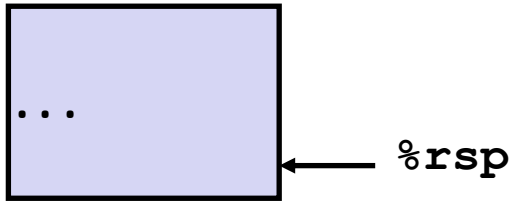
Updated Stack Structure




```
call_incr:
   subq     $16, %rsp
   movq     $15213, 8(%rsp)
   movq     $3000, %rsi
   leaq     8(%rsp), %rdi
   call     incr
   addq     8(%rsp), %rax
   addq     $16, %rsp
   ret
```

Register	Use(s)
`%rax`	Return value

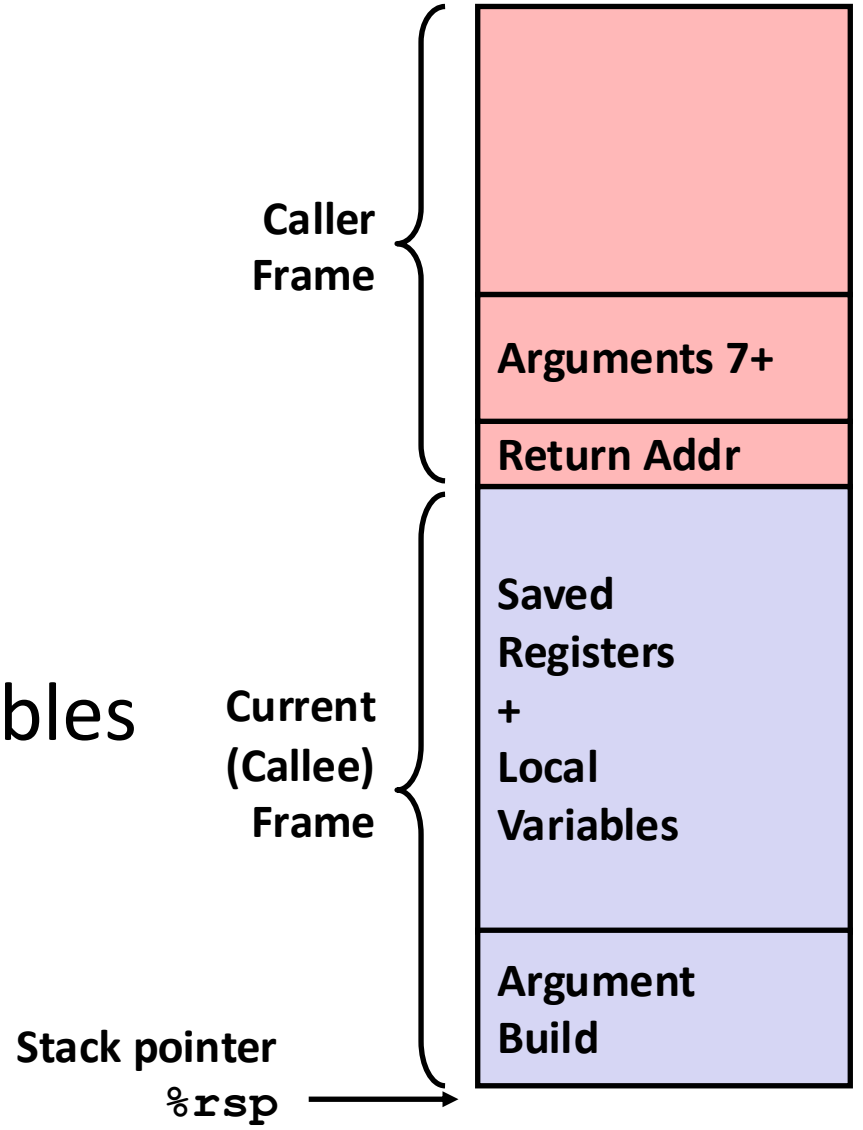
Final Stack Structure



Open Questions

- What are the initial values of variables created on the stack?

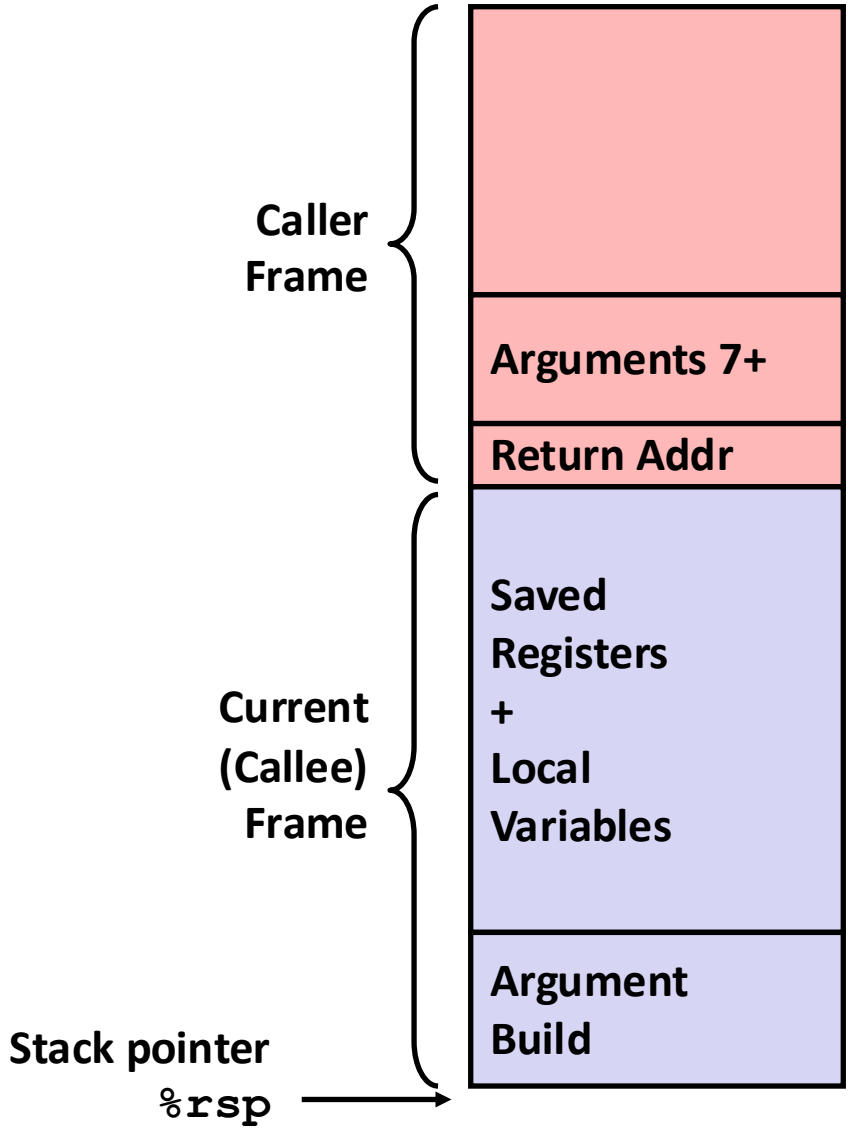
- Is there a limit to how many local variables a function can have?



Open Questions

- What are the initial values of variables created on the stack?
  - Undefined behavior in C
  - Machine just creates a variable in the stack
    - Initial value is whatever was there before

- Is there a limit to how many local variables a function can have?
  - Based on stack limit of the process
  - Stack keeps growing until it runs out of space
    - OS can do lots of tricks to give it more memory



Topics

- Procedures
  - Managing Local Data
- **Register saving**
  - Recursion Example

Register Saving

- Can a function use `%rdx` for temporary storage?

Caller

```
yo:
    • • •
    movq $15213, %rdx
    call who
    addq %rdx, %rax
    • • •
    ret
```

Callee

```
who:
    • • •
    subq $18213, %rdx
    • • •
    ret
```

  - Contents of register **`%rdx`** overwritten by **`who!`**
  - This could be trouble → something should be done!
    - Need some coordination

Reusing registers

- Problem: registers are shared between functions
  - Callee could overwrite caller's registers by accident

(function that's run) (code that's calling the function)

- How does each function know which registers are safe to use?

- Solution:
  - Save original register value to stack
  - Use register as needed
  - Restore original register value from stack

- New question: *when* should the saving happen? In advance or on demand?

Saving registers in advance

- New question: who should save the registers, Caller or Callee?

- Attempt 1: Save everything in advance
  - Caller knows which registers it is using
  - Before calling a function, save all registers it is going to need after the call

  - Downside: Caller doesn't know what Callee needs
    - Wasted stores to memory if Callee doesn't need those registers

  - Example: which registers does `printf()` need to use?

Saving registers on demand

- New question: who should save the registers, Caller or Callee?

- Attempt 2: Save everything on demand
  - Callee knows which registers it is using
  - At the start of a function, save all registers it is going to use

  - Downside: Callee doesn't know what Caller was using
    - Wasted stores to memory if Caller wasn't using those registers

  - Example: which registers does code that calls `printf()` use?

Compromise: some registers in advance, some on demand

- Neither the Caller nor the Callee has perfect knowledge of register usage/availability

- Designate certain registers are saved in certain way
  - Some are saved in advance: Caller saved
  - Some are saved on demand: Callee saved

- Remember: Caller and Callee are just designations for one call event
  - Functions can and do act as both at different times
  - If `A()` calls `B()` calls `C()`, then `B()` is both Callee of `A` and Caller of `C`

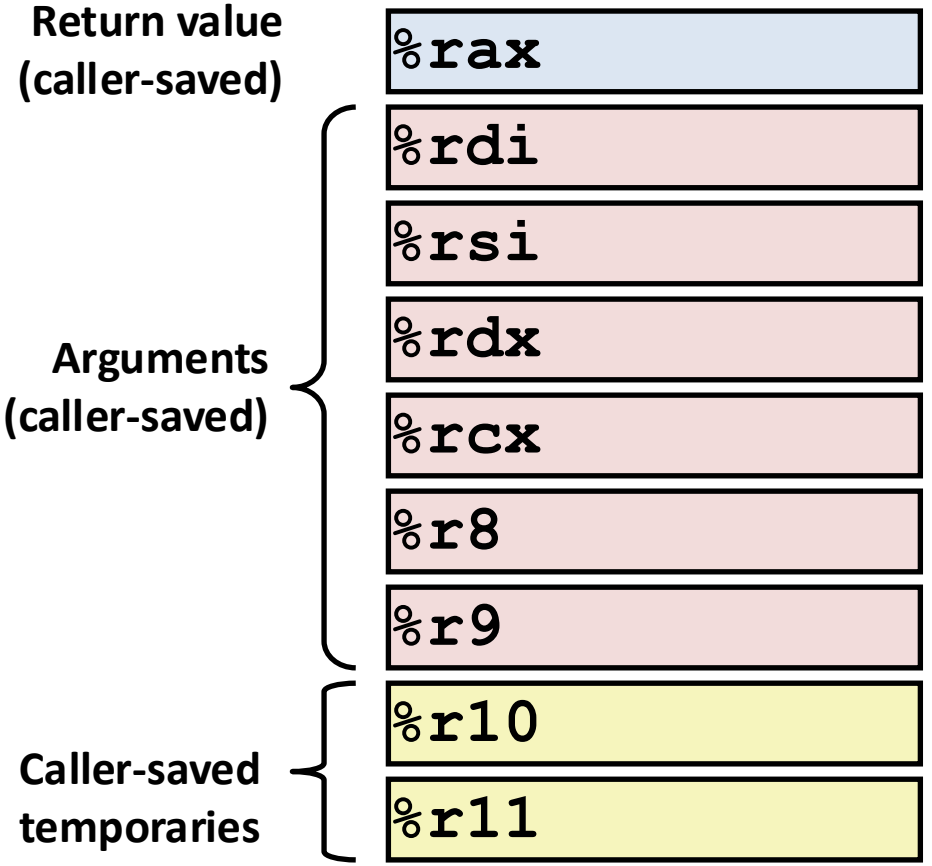
Full Rules for Register Saving

1. Does the function use any callee-saved (on-demand) registers?
   - They MUST be saved before use and restored before returning

2. Does the code call any functions?
   - If no, you're done
   - If yes: do any caller-saved (in-advance) registers need to keep their original value after the function call returns?
     - If no, you're done (if we don't need the registers we don't save them)
     - If yes, save them before the function call and restore them after it

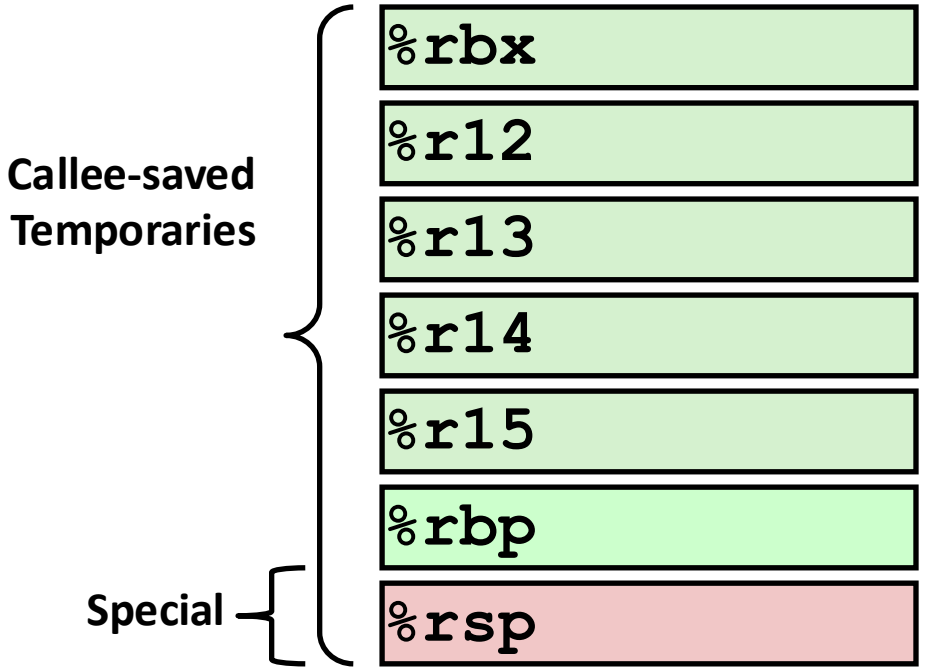
x86-64 Linux Register Usage #1 (caller-saved, in advance)

- `%rax`
  - Return value
  - Caller-saved
  - **Will** be modified by function we're about to call
- `%rdi`, …, `%r9`
  - Arguments
  - Caller-saved
  - Could be modified by function we're about to call
- `%r10`, `%r11`
  - Caller-saved
  - Could be modified by function we're about to call



x86-64 Linux Register Usage #2 (callee-saved, on demand)

- **`%rbx`, `%rbp`, `%r12-%r15`**
  - Callee-saved
  - Every function must save/restore the original values if it wants to use these registers



- **`%rsp`**
  - Special form of callee-saved
  - Always restored to original value upon exit from procedure
    - The current stack frame is removed before returning

x86-64 Integer Registers: Usage Conventions

Caller Saved	In advance
Callee saved	On demand

`%rax`	Return value
`%rbx`	Callee saved
`%rcx`	Argument #4
`%rdx`	Argument #3
`%rsi`	Argument #2
`%rdi`	Argument #1
`%rsp`	Stack pointer
`%rbp`	Callee saved

`%r8`	Argument #5
`%r9`	Argument #6
`%r10`	Caller saved
`%r11`	Caller Saved
`%r12`	Callee saved
`%r13`	Callee saved
`%r14`	Callee saved
`%r15`	Callee saved

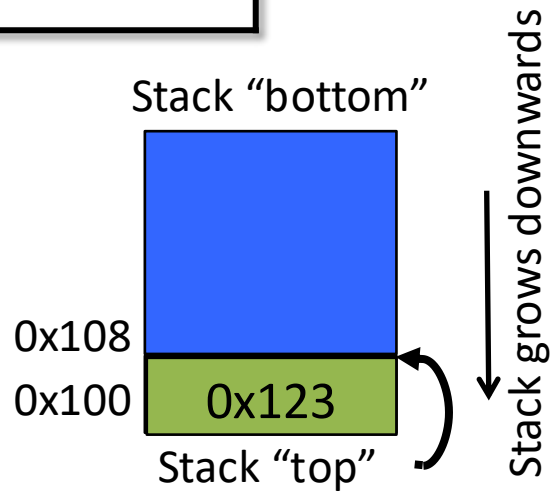
Push and Pop instructions

Instruction	Effect	Description
`pushq` S	R [%rsp] ← R [%rsp] – 8; M [R[%rsp]] ← S	Store S onto the stack
`popq` D	D ← M [R[%rsp]] R [%rsp] ← R [%rsp] + 8;	Retrieve D from the stack

- Example:

```
%rax = 0x123, %rdx = 0x0, %rsp = 0x108

pushq %rax
popq %rdx          %rsp = 0x100
                   %rdx = 0x123; %rsp = 0x108
```



- Remember, stack is just memory
  - Can also use memory moves and modify `%rsp` manually!
  - Functions often mix the two, push some registers and allocate extra space

Saving a register to the stack

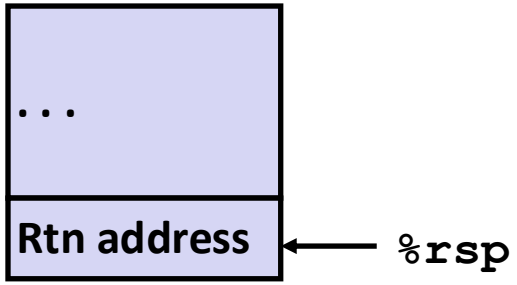
```
long call_incr2(long x) {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

Still need **x** after the call!

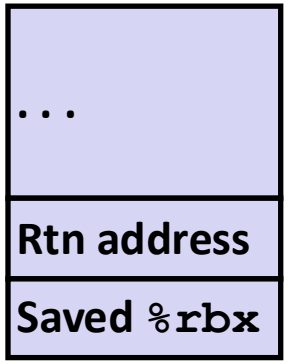
%rbx is callee-save (on demand)

```
call_incr2:
  pushq   %rbx
  subq    $16, %rsp
  movq    %rdi, %rbx
  movq    $15213, 8(%rsp)
  movq    $3000, %rsi
  leaq    8(%rsp), %rdi
  call    incr
  addq    %rbx, %rax
  addq    $16, %rsp
  popq    %rbx
  ret
```

Initial Stack Structure



Resulting Stack Structure



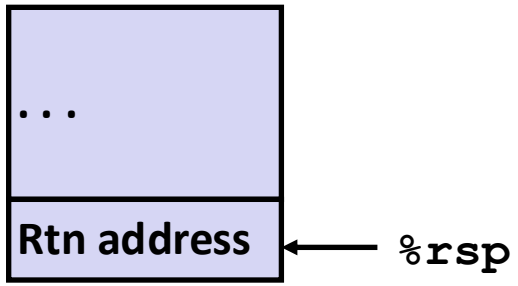
%rbx is callee-saved and we use it -> Save **%rbx**

Manually allocating stack space

```
long call_incr2(long x) {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

Still need **x** after the call!

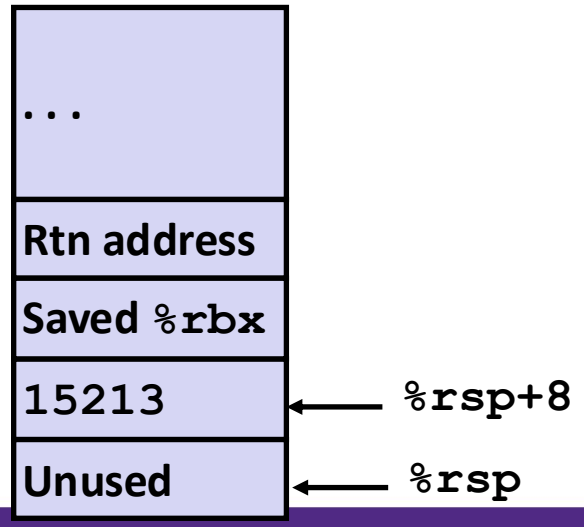
Initial Stack Structure



%rbx is callee-save (on demand)

```
call_incr2:
  pushq   %rbx
  subq    $16, %rsp
  movq    %rdi, %rbx
  movq    $15213, 8(%rsp)
  movq    $3000, %rsi
  leaq    8(%rsp), %rdi
  call    incr
  addq    %rbx, %rax
  addq    $16, %rsp
  popq    %rbx
  ret
```

Resulting Stack Structure



FYI: Stack moves in multiples of 16 whenever possible.

This accommodates alignment for any 128-byte values on the stack.

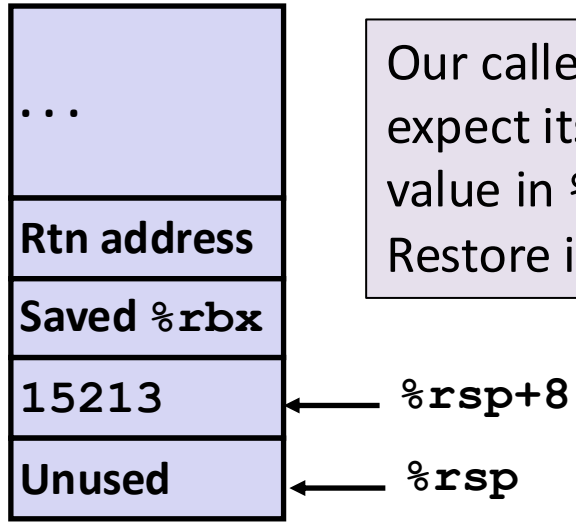
Restoring the stack and register before a return

```
long call_incr2(long x) {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

%rbx is callee-save (on demand)

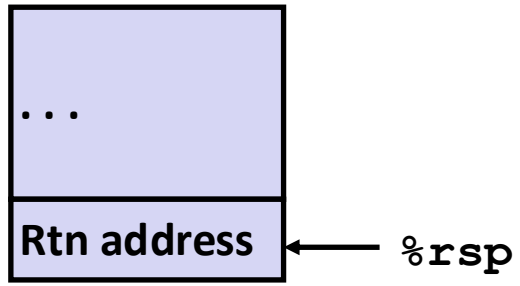
```
call_incr2:
  pushq   %rbx
  subq    $16, %rsp
  movq    %rdi, %rbx
  movq    $15213, 8(%rsp)
  movq    $3000, %rsi
  leaq    8(%rsp), %rdi
  call    incr
  addq    %rbx, %rax
  addq    $16, %rsp
  popq    %rbx
  ret
```

Resulting Stack Structure



Our caller can expect its own value in **%rbx** Restore it!

Pre-return Stack Structure



Topics

- Procedures
  - Managing Local Data
- Register saving
  - **Recursion Example**

Recursive Function

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

```
pcount_r:
  movq    $0, %rax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andq    $1, %rbx
  shrq    %rdi # (by 1)
  callq   pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

Note: `rep` instruction inserted as no-op. You can ignore it.

Recursive Function Base Case

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

```
pcount_r:
  movq    $0, %rax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andq    $1, %rbx
  shrq    %rdi # (by 1)
  callq   pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

Checks if `%rdi` is zero

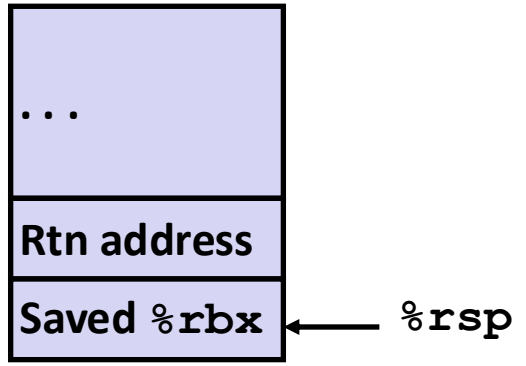
Register	Use(s)	Type
%rdi	x	Argument
%rax	Return value	Return value

Recursive Function Register Save

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

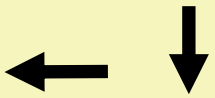
```
pcount_r:
  movq    $0, %rax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andq    $1, %rbx
  shrq    %rdi # (by 1)
  callq   pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

Register	Use(s)	Type
%rdi	x	Argument



Recursive Function Call Setup

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```



```
pcount_r:
  movq    $0, %rax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andq    $1, %rbx
  shrq    %rdi # (by 1)
  callq   pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

Register	Use(s)	Type
%rdi	x >> 1	Rec. argument
%rbx	x & 1	Callee-saved

Recursive Function Call

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```



```
pcount_r:
  movq    $0, %rax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andq    $1, %rbx
  shrq    %rdi # (by 1)
  callq   pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

Register	Use(s)	Type
%rbx	x & 1	Callee-saved
%rax	Recursive call return value	

Recursive Function Result

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```



```
pcount_r:
  movq    $0, %rax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andq    $1, %rbx
  shrq    %rdi # (by 1)
  callq   pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

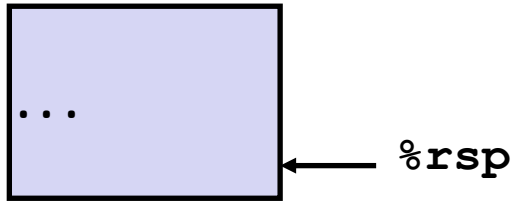
Register	Use(s)	Type
%rbx	x & 1	Callee-saved
%rax	Return value	

Recursive Function Completion

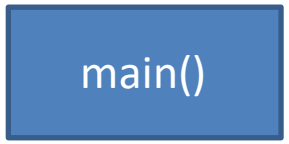
```
/* Recursive popcount */
long pcount_r(unsigned long x) {
  if (x == 0)
    return 0;
  else
    return (x & 1)
           + pcount_r(x >> 1);
}
```

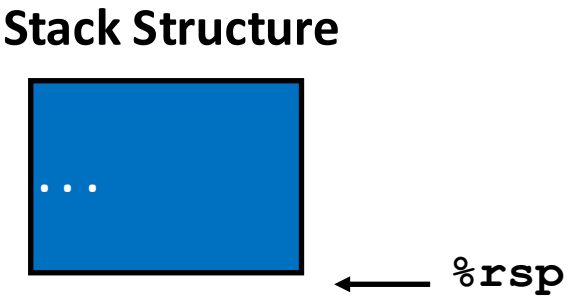
```
pcount_r:
  movq    $0, %rax
  testq   %rdi, %rdi
  je      .L6
  pushq   %rbx
  movq    %rdi, %rbx
  andq    $1, %rbx
  shrq    %rdi # (by 1)
  callq   pcount_r
  addq    %rbx, %rax
  popq    %rbx
.L6:
  rep; ret
```

Register	Use(s)	Type
%rax	Return value	Return value

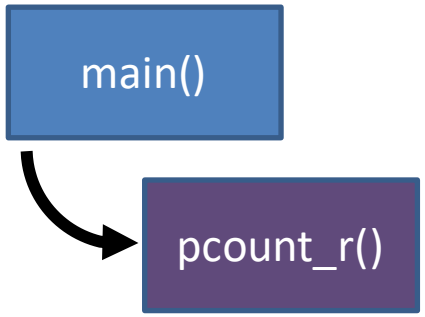


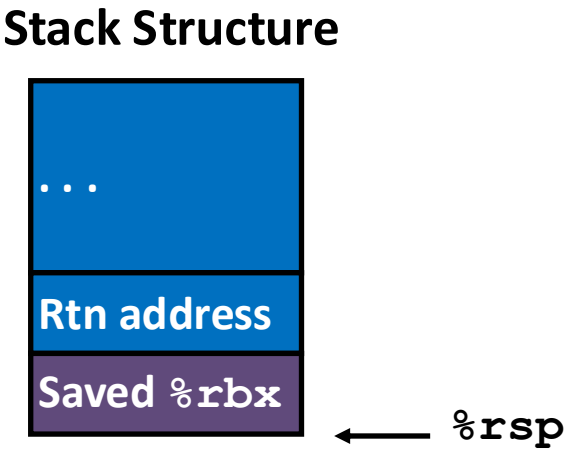
Example three recursions in



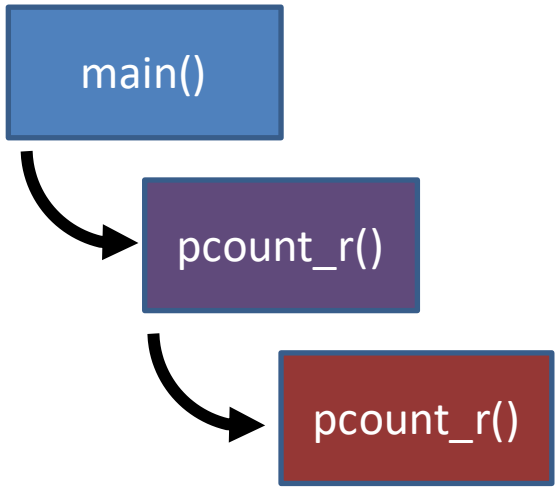


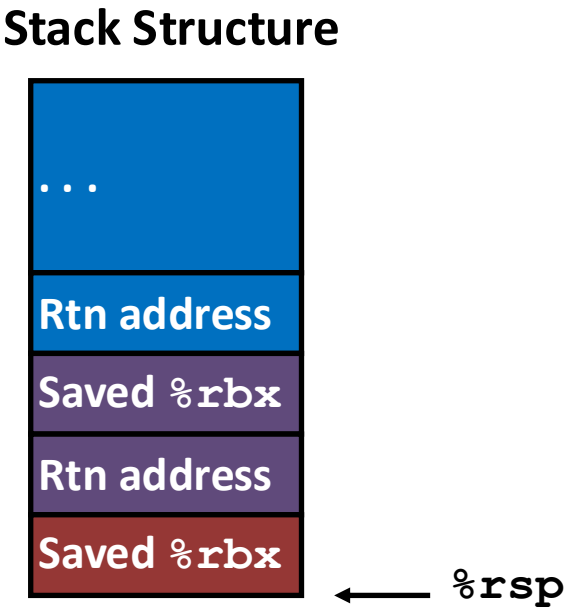
Example three recursions in



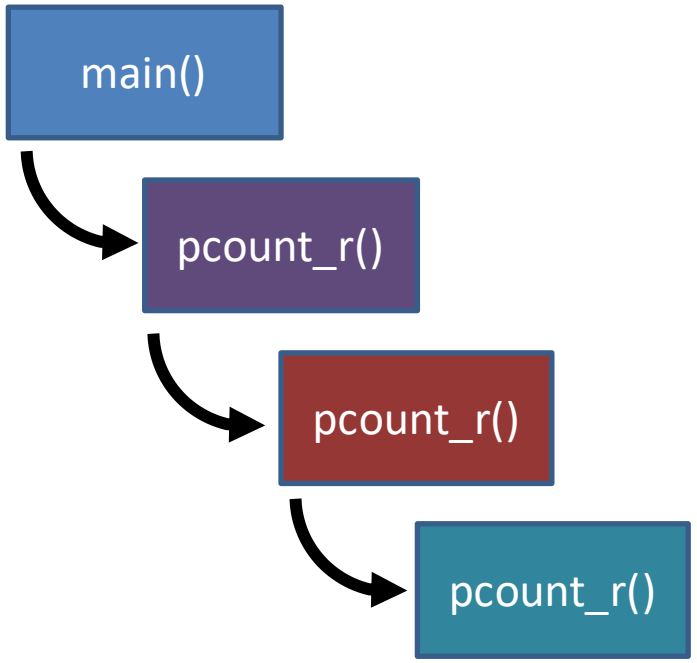


Example three recursions in

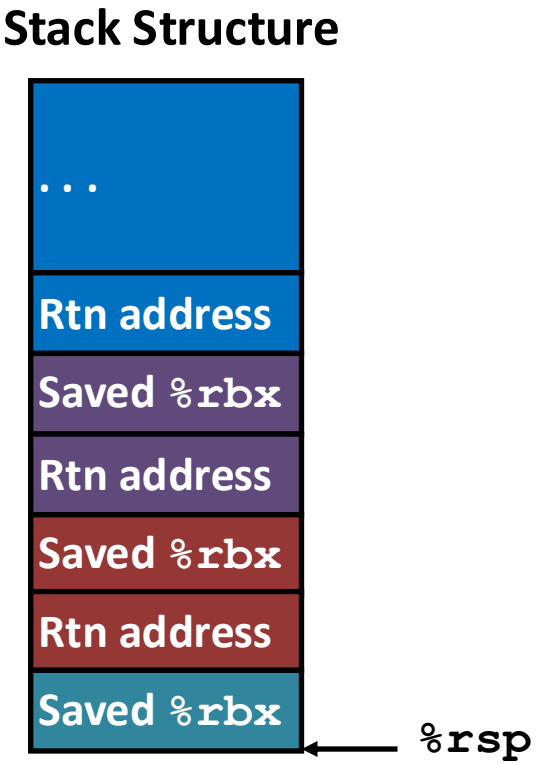




Example three recursions in



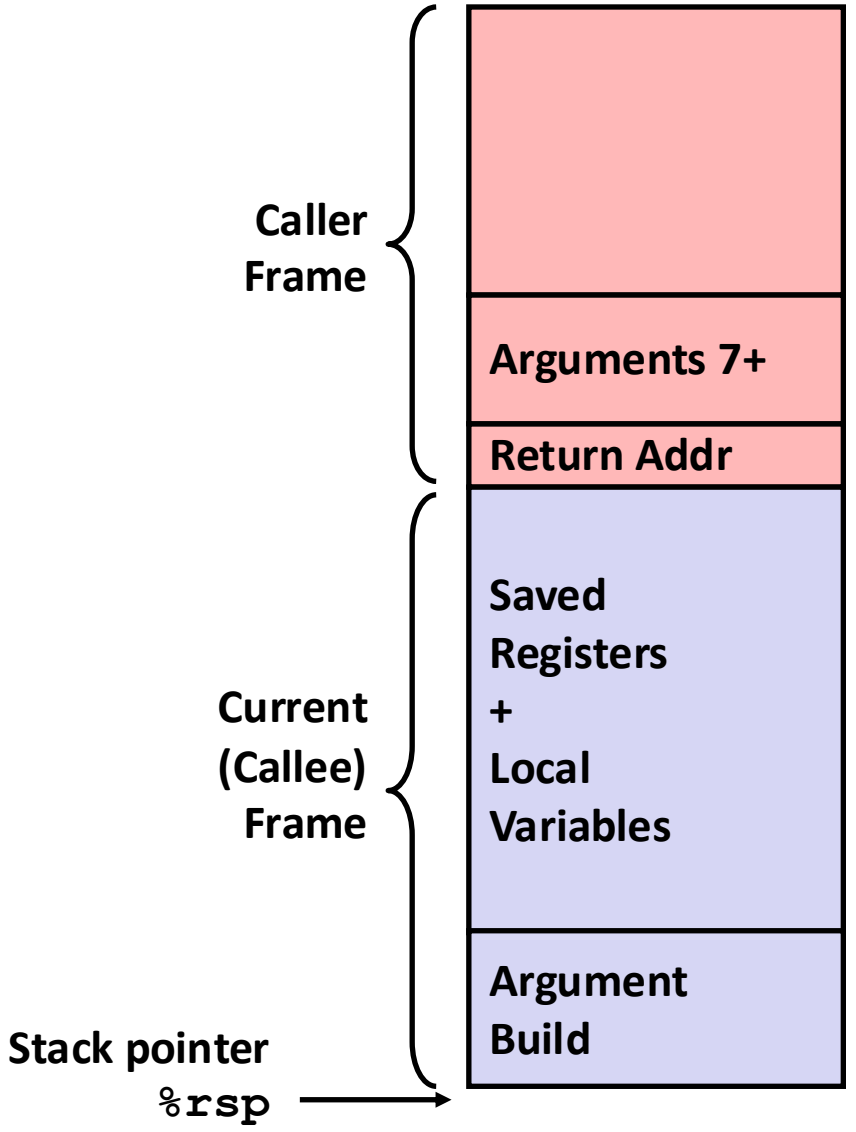
Executing, but has not yet called pcount_r() again



x86-64 Procedure Summary

- Important Points
  - The stack holds local function data
  - Builds and cleans like a stack data structure
  - The stack makes recursion work

- Calling convention
  - Caller-saved registers saved **in advance** before call
  - Put arguments in registers (1-6)
  - Put further arguments on top of stack (7+)
  - Put return address on top of stack
  - Callee can safely store values in local stack frame and in callee-saved registers (after saving them)
  - Result return in **`%rax`** and restore callee-saved registers before returning



Topics

- Procedures
  - Managing Local Data
- Register saving
  - Recursion Example
- Computer Security
  - **Buffer Overflows**

Memory Referencing Bug Example

```
typedef struct {
  int a[2];
  double d;
} struct_t;

double fun(int i) {
  volatile struct_t s; // volatile ≈ don't optimize this away
  s.d = 3.14;
  s.a[i] = 1073741824; // Possibly out of bounds
  return s.d;
}
```

```
fun(0)  ☞  3.14
fun(1)  ☞  3.14
fun(2)  ☞  3.1399998664856
fun(3)  ☞  2.00000061035156
fun(4)  ☞  3.14
fun(5)  ☞  3.14
fun(6)  ☞  Segmentation fault (core dumped)
```

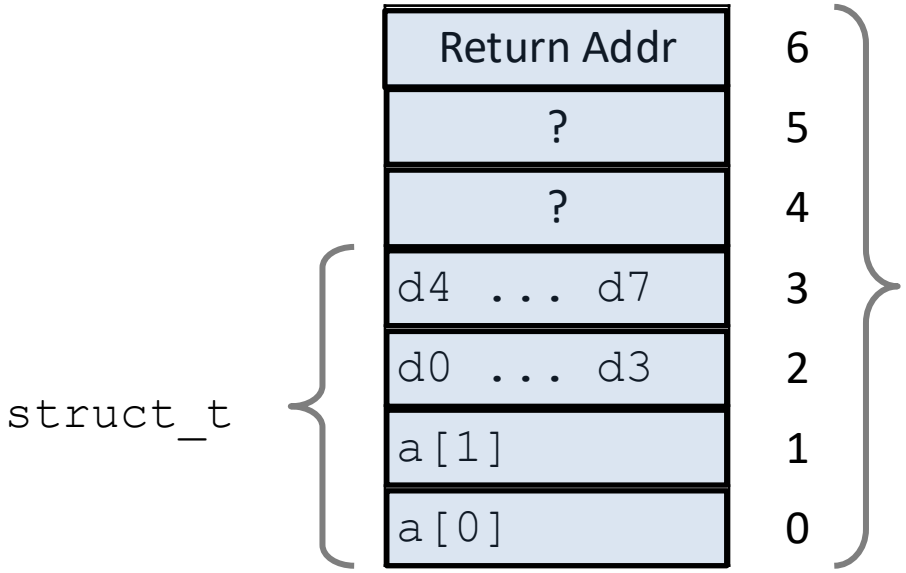
- Abuses undefined behavior
- Result is system specific

Memory Referencing Bug Example

```
typedef struct {
  int a[2];
  double d;
} struct_t;
```

fun(0)	☞	3.14
fun(1)	☞	3.14
fun(2)	☞	3.1399998664856
fun(3)	☞	2.00000061035156
fun(4)	☞	3.14
fun(5)	☞	3.14
fun(6)	☞	Segmentation fault

Explanation:



K	Types
1	char
2	short
4	int, float
8	long, double, char *

Such problems are a ***BIG*** deal

- Generally called a “buffer overflow”
  - Going past end of memory allocated for an array (AKA buffer)
- Why is it a big deal?
  - #1 ***technical*** cause of security vulnerabilities
    - (#1 overall cause is social engineering)
- Most common form:
  - Unchecked lengths on string inputs from a user
  - Particularly with character arrays on the stack
    - Sometimes referred to as “stack smashing”